Activity 2.4: Ensure modular code structure

- **/token**: Issues a secure JWT token after successful authentication, enabling authorized access to protected API routes.

```
@app.post("/token", response_model=Token)
async def login_for_access_token(form_data: OAuth2PasswordRequestForm = Depends()):
    """Login endpoint to get access token"""
    user = authenticate_user(users_db, form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=401,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )

    # Create access token
    access_token_expires = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = create_access_token(
        data={"sub": user.username}, expires_delta=access_token_expires
    )

    # Create or update session for the user
    existing_user_data = {}
    if user.username in active_sessions:
```

```
        existing_user_data = active_sessions[user.username].user_data
        # If there's an old token, blacklist it
        old_token = active_sessions[user.username].token
        blacklisted_tokens.add(old_token)

    active_sessions[user.username] = UserSession(
        username=user.username,
        login_time=datetime.utcnow(),
        last_activity=datetime.utcnow(),
        token=access_token,
        user_data=existing_user_data
    )

    # Return response with cookie
    response = JSONResponse(content={"access_token": access_token, "token_type":
"bearer"})
    response.set_cookie(
        key="access_token",
        value=access_token, # Store token directly without Bearer prefix
        httponly=True,
        max_age=ACCESS_TOKEN_EXPIRE_MINUTES * 60,
        samesite="lax"
    )
    return response
```

- **/session-info**: Retrieves metadata about the current user session, such as user ID and login status.
- **/session-data**: Returns detailed session-specific data used for personalization and recommendation tracking.

```
@app.get("/session-info")

async def get_session_info(request: Request, current_user: UserInDB =
Depends(get_current_active_user)):

    """Get current user's session information"""

    if current_user.username in active_sessions:

        session = active_sessions[current_user.username]

        return {

            "username": session.username,

            "login_time": session.login_time,

            "last_activity": session.last_activity,

            "session_duration": (datetime.utcnow() - session.login_time).total_seconds() // 60, #
in minutes

            "user_data": session.user_data

        }

    else:

        raise HTTPException(status_code=404, detail="No active session found")




@app.post("/session-data")

async def update_session_data(

    data: Dict[str, Any],
```

```
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Update user's session data"""
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data.update(data)
        active_sessions[current_user.username].last_activity = datetime.utcnow()
        return {"message": "Session data updated", "data":
active_sessions[current_user.username].user_data}
    else:
        raise HTTPException(status_code=404, detail="No active session found")
#Fix for the/home-budget endpoint to properly handle the form submission
```